BOOK TWO • LEVEL UP

PYTHON

Python Programming

The Coding-Round Gym:
Harder Problems from Core Python — with Concepts, Applications & Full Solutions

A Continuation of Book One — No DSA, Just Sharper Fundamentals

STRINGS • NUMBERS • COLLECTIONS • FUNCTIONS • RECURSION • APPLIED LOGIC

Before You Begin

> *This is Book Two. It assumes you have worked through Book One — variables, data types, loops, functions, classes, and exception handling. Here we do not re-teach those; we **stress-test** them.*

How this book is built

Every problem is wrapped with three learning aids so you understand the *why*, not just the *how*:

CONCEPT
The core idea or technique the problem is teaching — read this first if you are stuck.

WHERE IT IS USED
A real-world place the same technique shows up, so the pattern sticks in memory.

HINT A nudge toward the approach without giving away the full answer.

Try each problem yourself first. The **complete solutions** — code, output, and a step-by-step walkthrough — are collected at the very end of the book in the Solutions section, numbered to match (Solution 1.3 answers Problem 1.3).

Difficulty: most problems are MEDIUM, with a handful of HARD ones to push you. None require data-structure or algorithm theory — only solid core Python.

THE GOAL
Fluency under pressure. When these patterns — frequency counters, seen-sets, digit math, recursion, multi-key sorts — become automatic, a timed coding round feels routine.

A Method for Every Problem

> *Before the problem sets, here is the four-step method this whole book trains. Speed in a coding round comes from a repeatable process, not from luck. We will walk one problem through it end to end.*

The four steps

1. **Restate the goal** in one sentence and note the exact input and output shapes.
2. **Name the pattern.** Ask which technique fits — counting? a seen-set? digit math? recursion? (The Pattern Playbook at the back lists them all.)
3. **Handle the edge cases** up front: empty input, one element, duplicates, zero.
4. **Write the smallest correct version**, then trace one example by hand.

Worked example

Warm-up MEDIUM

Given a list of integers, return the **first value that appears more than once**, or None if all values are unique.

STEP 1 — RESTATE

Input: a list of ints. Output: the earliest value whose second occurrence appears soonest — really, the first value we encounter that we have seen before.

STEP 2 — NAME THE PATTERN

"Have I seen this before?" is the classic **seen-set** pattern. Keep a set of values encountered; the first value already in the set is the answer.

STEP 3 — EDGE CASES

Empty list → None. All unique → None. The set approach handles both naturally because we only return when we find a repeat.

Step 4 — Smallest correct version:

```python
def first_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:      # seen it already?
            return num
        seen.add(num)
    return None              # no duplicates

print(first_duplicate([3, 1, 4, 1, 5]))   # 1
print(first_duplicate([1, 2, 3]))         # None
```

```
Output ▸ 1
None
```

Hand-trace: we see 3 (add), 1 (add), 4 (add), then 1 again — already in `seen`, so return 1. For the all-unique list we fall through the loop and return None. That is the entire method: restate, name the pattern, cover edges, write and trace. Every problem ahead rewards the same discipline.

> USE THIS EVERYWHERE
>
> When a problem feels hard, it is usually because step 2 has not clicked yet. Skim the Pattern Playbook at the back, match the shape, and the code follows.

Table of Contents

GETTING STARTED

Before You Begin — how this book works

A Method for Every Problem (worked warm-up)

PROBLEM CHAPTERS

1. String Manipulation — 8 problems (6M / 2H)

2. Number Theory & Digit Math — 8 problems (5M / 3H)

3. Lists & Dictionaries — 7 problems (5M / 2H)

4. Matrices & Grids — 5 problems (3M / 2H)

5. Functions, Recursion & Beyond — 6 problems (1M / 5H)

6. Applied Mini-Problems — 7 problems (2M / 5H)

SOLUTIONS (AT THE END)

Solutions 1: String Manipulation — 8 worked answers

Solutions 2: Number Theory & Digit Math — 8 worked answers

Solutions 3: Lists & Dictionaries — 7 worked answers

Solutions 4: Matrices & Grids — 5 worked answers

Solutions 5: Functions, Recursion & Beyond — 6 worked answers

Solutions 6: Applied Mini-Problems — 7 worked answers

APPENDIX

Pattern Playbook & Final Checklist

1. String Manipulation

Text is everywhere in coding rounds. These problems push past basic slicing into compression, validation, rotation, and frequency analysis — all with plain Python.

Problem 1.1

MEDIUM

Given a sentence, return it with the word order reversed but each word's letters intact. Example: **"code every day" → "day every code"**.

CONCEPT: SPLIT → REVERSE LIST → JOIN

Any word-level task starts by turning the string into a list of words with `.split()`. A list can be reversed with slicing `[::-1]`, then re-joined with `' '.join()`. Reversing *words* is different from reversing *characters* — keep that distinction sharp.

WHERE IT IS USED

This is the exact logic behind 'reverse name order' features (First Last → Last First) and reformatting addresses.

HINT Three chained operations, one line is enough.

Problem 1.2

MEDIUM

Check whether a string is a **pangram** — it contains every letter of the English alphabet at least once (case-insensitive).

CONCEPT: SETS FOR COVERAGE CHECKS

When a question asks 'does it contain all of X', think **sets**. Build a set of the letters present, then compare against the full alphabet set. Set operations like `<=` (subset) answer 'is everything covered?' in one step.

WHERE IT IS USED

Pangram/coverage logic is used to validate that a form collected all required field types, or that a test suite touched every category.

HINT `set(string)` plus the 26 letters. A subset test finishes it.

Problem 1.3

HARD

Compress a string using **run-length encoding**: replace runs of repeated characters with the character followed by its count. Example: **"aaabbc" → "a3b2c1"**.

CONCEPT: TRACKING RUNS WITH A PREVIOUS-VALUE POINTER

To collapse consecutive duplicates you walk the string while remembering the current character and a running count. When the next character differs, you 'flush' the current run to the result and reset. This 'previous vs current' pattern is a workhorse for stream-processing problems.

WHERE IT IS USED

Run-length encoding is real compression, used in image formats (BMP, TIFF) and to shrink repetitive log data before storage.

HINT Loop once, keep `count` and `prev`. Do not forget to flush the final run after the loop.

Problem 1.4

MEDIUM

Given a string, find the character(s) with the highest frequency. If several tie, return them all in the order they first appear.

CONCEPT: FREQUENCY DICT + A SECOND FILTERING PASS

Counting with `dict.get(k, 0) + 1` gives frequencies. The twist is handling **ties**: first find the maximum count, then collect every key matching it. Preserving first-appearance order means iterating the original string, not the dictionary.

WHERE IT IS USED

'Top result' logic with tie-handling appears in analytics dashboards, voting systems, and 'most-used tag' features.

HINT Two passes: one to count, one over the original string to collect maxes without repeats.

Problem 1.5

HARD

Validate a string as a simple **identifier**: it must start with a letter or underscore, contain only letters, digits, or underscores, and not be a Python keyword. Return True/False.

CONCEPT: LAYERED VALIDATION WITH EARLY EXITS

Validation problems are cleanest when you check one rule at a time and return `False` as soon as any fails. Combining character-class methods (`.isalpha()` , `.isalnum()`) with a keyword set covers all rules without a regex.

WHERE IT IS USED

This is literally how compilers and linters decide whether a variable name is legal — the same idea guards usernames and column names.

HINT Check emptiness, then the first character, then the rest, then the keyword set. `keyword.kwlist` has all keywords.

Problem 1.6

MEDIUM

Given two strings, find the **longest common prefix** they share. Example: **"flower", "flight"** → **"fl"**.

CONCEPT: PARALLEL SCANNING WITH ZIP

`zip` pairs characters from both strings position by position and stops at the shorter one automatically. Walk the pairs until they differ; everything before that point is the shared prefix. `zip` removes all the index bookkeeping.

WHERE IT IS USED

Longest-common-prefix logic powers autocomplete and file-path grouping.

HINT Loop `zip(a, b)` ; break at the first mismatch, collecting matches.

Problem 1.7

MEDIUM

Determine whether one string is a **rotation** of another. Example: **"waterbottle"** is a rotation of **"erbottlewat"**.

CONCEPT: THE DOUBLED-STRING TRICK

Any rotation of a string is a substring of that string concatenated with itself. So `b` is a rotation of `a` exactly when lengths match and `b in a + a`. One elegant membership test replaces a rotation loop.

WHERE IT IS USED

Rotation checks appear in cyclic-sequence matching and circular-buffer comparisons.

HINT Check equal length, then `b in (a + a)`.

Problem 1.8

MEDIUM

Given a string with words separated by spaces, return a dictionary mapping each word length to the list of words having that length.

CONCEPT: GROUPING BY A COMPUTED KEY

Grouping means bucketing items by some property — here, length. `setdefault(key, [])` creates the bucket on first sight, then you append. The key can be any computed value, not just the item itself.

WHERE IT IS USED

Grouping-by-attribute is everywhere: files by extension, orders by status, students by grade band.

HINT Key each word by `len(word)` into a dict of lists.

2. Number Theory & Digit Math

Working with numbers as digits, factors, and bases. No libraries — just arithmetic with `%`, `//`, and clean loops.

Problem 2.1

MEDIUM

Given a positive integer, return the count of digits, the sum of digits, and the reversed number — without converting to a string.

CONCEPT: DIGIT EXTRACTION WITH % AND //

To take a number apart mathematically, `n % 10` gives the last digit and `n // 10` removes it. Repeating until `n` hits 0 walks every digit. This arithmetic approach is often required when string conversion is disallowed.

WHERE IT IS USED

Digit math powers checksum algorithms (like credit-card validation) and converting between number bases.

HINT Loop `while n > 0`. Build the reverse as `rev = rev*10 + digit`.

Problem 2.2

MEDIUM

Print the first **n** prime numbers (not primes up to n, but exactly n primes). Example for n=5: **2, 3, 5, 7, 11**.

CONCEPT: SEPARATING A TEST FROM A GENERATOR LOOP

Reuse a small `is_prime` helper, then keep a candidate counter climbing and collect primes until you have `n` of them. Splitting the 'test one number' logic from the 'keep going until I have enough' loop keeps each part simple.

WHERE IT IS USED

Prime generation underlies cryptographic key creation and hashing table sizing.

HINT A `while len(primes) < n` loop with a rising candidate is cleaner than guessing an upper bound.

Problem 2.3

HARD

Convert an integer to its **binary representation** as a string, without using `bin()` or format specifiers.

CONCEPT: REPEATED DIVISION BY THE TARGET BASE

Any base conversion uses the same recipe: repeatedly divide by the base and record remainders, which form the digits in **reverse** order. For binary the base is 2, so remainders are 0/1. Reversing the collected remainders gives the answer.

WHERE IT IS USED

Understanding base conversion demystifies how computers store all data and how hexadecimal color codes and IP addresses work.

HINT Collect `n % 2` each step, do `n //= 2`, then reverse. Handle `n == 0` as a special case.

Problem 2.4

MEDIUM

A number is a **perfect number** if it equals the sum of its proper divisors (divisors excluding itself). Example: 6 = 1+2+3. Find all perfect numbers up to a limit.

CONCEPT: DIVISOR SUMMATION WITH A BOUNDED LOOP

To find proper divisors of `n`, test each candidate from 1 to `n-1` (or up to `n//2` for efficiency) and keep those that divide evenly. Summing them and comparing to `n` classifies the number.

WHERE IT IS USED

Perfect numbers are a classic number-theory exercise; the divisor-sum technique generalizes to 'sum of factors' problems in finance and scheduling.

HINT Divisors above `n//2` (except n itself) can't divide n evenly, so stop there.

Problem 2.5

HARD

Given an amount and a list of coin denominations, compute the **minimum number of coins** needed using a greedy approach (assume standard denominations like [1, 2, 5, 10, 20, 50]).

CONCEPT: GREEDY SELECTION WITH DIVMOD

A greedy strategy always takes the largest coin that fits, then repeats on the remainder. `divmod(amount, coin)` returns how many of that coin fit and what's left over in one step. Sorting denominations high-to-low makes 'largest first' natural.

WHERE IT IS USED

This is exactly how cash registers and ATMs dispense change with the fewest notes and coins.

HINT Sort coins descending. For each, `count, amount = divmod(amount, coin)`.

Problem 2.6

MEDIUM

Given two integers, compute their **LCM** (least common multiple) using their GCD.

CONCEPT: BUILDING LCM FROM GCD

LCM and GCD are linked: `lcm(a,b) = a * b // gcd(a,b)`. Reusing a small GCD helper (Euclid's algorithm) gives LCM almost for free. Recognizing that one quantity can be derived from another saves work.

WHERE IT IS USED

LCM schedules repeating events that must align (e.g. two tasks running every N and M minutes meeting again).

HINT Write `gcd` with Euclid, then divide the product by it.

Problem 2.7

MEDIUM

Count how many integers from 1 to n are **divisible by 3 or 5**, and return their sum.

CONCEPT: FILTERING A RANGE WITH A COMPOUND CONDITION

A single pass over `range(1, n+1)` with an `or` condition handles 'divisible by 3 or 5'. A comprehension collects the matches; `len` and `sum` summarize them. Clear conditions beat clever math when readability matters.

WHERE IT IS USED

This 'FizzBuzz-sum' style filtering models tallying qualifying records (e.g. orders over a threshold in certain regions).

HINT `[x for x in range(1, n+1) if x%3==0 or x%5==0]`.

Problem 2.8

HARD

Given a number, return its **Roman numeral** representation (valid up to 3999).

CONCEPT: GREEDY SUBTRACTION WITH ORDERED VALUE/SYMBOL PAIRS

List the Roman values (including the subtractive pairs like 9=IX, 40=XL) from largest to smallest. Greedily subtract the largest value that fits, appending its symbol, until the number reaches 0. Ordered pairs make the greedy loop clean.

WHERE IT IS USED

Beyond numerals, greedy 'take the biggest chunk that fits' logic drives change-making, scheduling, and packing heuristics.

HINT Keep parallel lists of values and symbols including 900/400/90/40/9/4.

3. Lists & Dictionaries

The containers you'll reach for most. These sharpen prefix/suffix thinking, multi-key sorting, the 'seen set' complement trick, and inverted indexes.

Problem 3.1

HARD

Given a list of numbers, return a new list where each element is the product of all **other** elements (do not use division). Example: **[1,2,3,4] → [24,12,8,6]**.

CONCEPT: PREFIX AND SUFFIX PASSES

Without division, the trick is two sweeps: one left-to-right building the product of everything *before* each index, and one right-to-left for everything *after*. Multiplying the two gives 'product of all others'. Building results in passes avoids a slow nested loop.

WHERE IT IS USED

This prefix/suffix idea appears in running-total dashboards and in computing 'everything except me' aggregates in reports.

HINT First fill `result[i]` with the running product to its left, then multiply in the running product from the right.

Problem 3.2

MEDIUM

Merge two dictionaries of category → list-of-items so shared categories have their lists combined (no duplicate items, order preserved).

CONCEPT: SETDEFAULT FOR GROUPED MERGES

When values are lists, merging means extending rather than overwriting. `setdefault(key, [])` guarantees a list exists before you extend it. Guarding with a membership check keeps items unique while preserving first-seen order.

WHERE IT IS USED

Combining tagged data from two sources — e.g. merging playlists or search-result buckets — uses exactly this pattern.

HINT Copy the first dict's lists, then loop the second dict and append only new items.

Problem 3.3

MEDIUM

Given a list of (name, score) tuples, return the names sorted by score descending, and for equal scores, alphabetically by name.

CONCEPT: MULTI-KEY SORTING WITH TUPLE KEYS

A sort `key` can return a **tuple**; Python compares the first element, then the second on ties, and so on. To sort one field descending and another ascending, negate the numeric field (`-score`) while leaving the string ascending.

WHERE IT IS USED

Leaderboards, search rankings, and 'sort by price then rating' filters all rely on multi-key sorts.

HINT `key=lambda p: (-p[1], p[0])` gives score-desc, name-asc.

Problem 3.4

MEDIUM

Rotate a list to the right by **k** positions. Example: **[1,2,3,4,5]** rotated by 2 → **[4,5,1,2,3]**.

CONCEPT: SLICING WITH A MODULO-NORMALIZED K

Rotation is just slicing at the right split point. The last `k` elements move to the front: `lst[-k:] + lst[:-k]` . Using `k % len(lst)` first handles rotations larger than the list without extra work.

WHERE IT IS USED

Rotating buffers appear in scheduling round-robins, carousels/sliders, and cyclic data windows.

HINT Normalize `k` with modulo, then concatenate two slices.

Problem 3.5

MEDIUM

Given a list of integers and a target sum, return the first **pair** of values that add up to the target. Return None if no pair exists.

CONCEPT: 'SEEN' SET FOR COMPLEMENT LOOKUPS

Instead of checking every pair (slow), remember numbers you've already seen in a set. For each new number, its needed partner is `target - number` ; if that partner is already in the set, you've found the pair in a single pass.

WHERE IT IS USED

Complement-lookup logic drives 'find two transactions that sum to X' fraud checks and pricing-bundle finders.

HINT For each number compute the complement and check membership before adding it to the set.

Problem 3.6

MEDIUM

Given a list, return a dictionary mapping each element to a list of the **indexes** where it appears. Example: **[a,b,a,c,b] → {a:[0,2], b:[1,4], c:[3]}**.

CONCEPT: ENUMERATE + GROUPED SETDEFAULT

Pair `enumerate` (to get index and value together) with `setdefault(value, [])` to accumulate positions. This 'inverse index' turns a list into a lookup of where each value lives — a building block for search features.

WHERE IT IS USED

Building an inverted index like this is the core idea behind search engines and 'find all occurrences' tools.

HINT Loop `enumerate(lst)` ; append the index into the value's bucket.

Problem 3.7

HARD

Given two lists that may contain duplicates, return their **intersection** preserving how many times each element appears in both. Example: **[1,2,2,3]** and **[2,2,4]** → **[2,2]**.

CONCEPT: MULTISET INTERSECTION VIA COUNTS

A plain set intersection loses duplicates. To keep counts, count each list into a dict, then for shared keys take the **minimum** of the two counts — that's how many appear in both. This 'multiset' thinking handles quantity-aware overlaps.

WHERE IT IS USED

Multiset intersection matters in inventory reconciliation and comparing ingredient lists against what's in stock.

HINT Count both lists; for each shared key, emit it `min(c1, c2)` times.

4. Matrices & Grids

2D data with pure Python — no numpy. Transposes, rotations, diagonals, and magic squares that rely on index relationships and the `zip(*grid)` trick.

Problem 4.1

MEDIUM

Given a 2D grid (list of lists), return its **transpose** (rows become columns) without using external libraries.

CONCEPT: ZIP(*GRID) UNPACKS ROWS INTO COLUMNS

Unpacking a grid with `zip(*grid)` feeds each row as a separate argument to `zip`, which then pairs up the first elements, second elements, and so on — exactly a transpose. The `*` splat is the key trick.

WHERE IT IS USED

Transposing underlies switching between row-major and column-major views in spreadsheets and data frames.

HINT `[list(col) for col in zip(*grid)]`.

Problem 4.2

MEDIUM

Given a square matrix, compute the sum of both **diagonals** (count the center once if the size is odd).

CONCEPT: INDEX RELATIONSHIPS IN A GRID

In an n x n grid, the main diagonal is cells where row == col; the anti-diagonal is where row + col == n-1. Summing both while subtracting the shared center (odd n) avoids double counting. Recognizing index formulas replaces nested searching.

WHERE IT IS USED

Diagonal sums appear in image kernels, game-board win checks, and matrix property tests.

HINT Loop `i` from 0 to n-1; add `m[i][i]` and `m[i][n-1-i]`; fix the center for odd n.

Problem 4.3

MEDIUM

Given a grid of 0s and 1s, find the row with the **most** 1s and return its index (first such row on ties).

CONCEPT: TRACK BEST-SO-FAR WHILE SCANNING

Finding a maximum over rows needs only one pass with two variables: the best count seen and its index. `sum(row)` counts 1s directly since they're the only nonzero value. Updating 'best' only on a strict improvement keeps the first tie.

WHERE IT IS USED

'Row with most hits' logic drives finding the busiest hour, the top-selling region, or the densest area in a heatmap.

HINT `sum(row)` counts the 1s; keep `best_count` and `best_index`.

Problem 4.4

HARD

Rotate a square matrix 90° **clockwise** in place-style (return a new grid). Example first row becomes the last column.

CONCEPT: TRANSPOSE THEN REVERSE EACH ROW

A 90° clockwise rotation equals two simple operations: transpose the matrix (rows ↔ columns), then reverse each row. Breaking a complex transform into two known steps is a powerful problem-solving habit.

WHERE IT IS USED

Image editors rotate photos with exactly this operation; games rotate board pieces the same way.

HINT `zip(*m)` transposes; then reverse each resulting row.

Problem 4.5

HARD

Given a grid, check whether it is a **magic square**: all rows, all columns, and both diagonals sum to the same value.

CONCEPT: COLLECTING ALL LINE-SUMS INTO A SET

Compute every row sum, column sum, and diagonal sum, then put them all in a set. If the grid is magic, the set has exactly one element. Reducing 'are all these equal?' to 'does the set have size 1?' is a clean, general test.

WHERE IT IS USED

Validation-by-uniform-sum appears in checksum grids and puzzle validators.

HINT Gather row sums, column sums (via `zip(*m)`), and both diagonals; check `len(set(sums)) == 1`.

5. Functions, Recursion & Beyond

Functions as tools and as values — recursion for unknown depth, memoization for speed, composition, decorators, and generators that hold state.

Problem 5.1

HARD

Write a function that flattens an arbitrarily nested list of integers into a single flat list, to **any** depth. Example: **[1,[2,[3,[4]]],5]** → **[1,2,3,4,5]**.

CONCEPT: RECURSION FOR UNKNOWN DEPTH

When nesting can be any depth, recursion is the natural tool: for each element, if it's a list, flatten it (a recursive call) and extend; otherwise append it. Each call handles one layer and trusts itself to handle the rest.

WHERE IT IS USED

Flattening shows up when normalizing deeply nested JSON from APIs or config files into flat records.

HINT Use `isinstance(x, list)` to decide recurse-vs-append. `extend` the recursive result.

Problem 5.2

HARD

Implement a simple **memoized** Fibonacci using a dictionary cache so repeated calls are fast. Show that fib(50) returns instantly.

CONCEPT: CACHING RESULTS TO AVOID RECOMPUTATION

Plain recursive Fibonacci recomputes the same values exponentially. A **cache** (dict) stores each result the first time it's computed; subsequent calls read from the cache. This turns an unusable recursion into a fast one — the essence of dynamic programming, done with just a dict.

WHERE IT IS USED

Memoization speeds up any expensive pure function: currency conversions, layout calculations, or repeated database-free lookups.

HINT Pass a `cache={}` default or use a module-level dict; check it before recursing.

Problem 5.3 HARD

Write a function `compose(*funcs)` that returns a new function applying the given functions right-to-left. Example: `compose(double, increment)(5)` computes double(increment(5)) = 12.

CONCEPT: FUNCTIONS AS FIRST-CLASS VALUES

In Python, functions can be passed around and returned like any value. **Composition** builds a pipeline: the output of one becomes the input of the next. Returning an inner function that loops over the given functions creates reusable transformation chains.

WHERE IT IS USED

Function composition is the backbone of data pipelines and middleware — chaining validators, formatters, or transforms into one callable.

HINT Return an inner function; loop the funcs in reverse, feeding each result into the next.

Problem 5.4 MEDIUM

Build a function that takes a variable number of keyword arguments and returns them formatted as a query string. Example: `query(page=2, sort="name")` → **"page=2&sort=name"**.

CONCEPT: **KWARGS AND JOINING PAIRS

`**kwargs` collects keyword arguments into a dictionary. Turning a dict into `key=value` segments and joining them with a separator is a reusable formatting pattern — the same idea builds CSV rows or log lines.

WHERE IT IS USED

This is essentially how HTTP libraries assemble URL query strings from named parameters.

HINT Build a list of `f"{k}={v}"` then `"&".join(...)`.

Problem 5.5

HARD

Write a **decorator** that times how long a function takes and prints it, then returns the function's result unchanged.

CONCEPT: DECORATORS WRAP BEHAVIOR AROUND A FUNCTION

A decorator is a function that takes a function and returns a new one adding behavior before/after the original. The inner wrapper calls the real function, measures time around it, and returns its result. Decorators keep cross-cutting concerns (timing, logging) out of the core logic.

WHERE IT IS USED

Timing and logging decorators are standard in production code for performance monitoring without cluttering business functions.

HINT Return a `wrapper(*args, **kwargs)` that records time around `func(*args, **kwargs)`.

Problem 5.6

HARD

Implement a function that returns a **running average** generator: each time you send it a number, it yields the average of all numbers seen so far.

CONCEPT: GENERATORS HOLD STATE BETWEEN YIELDS

A generator function (using `yield`) pauses and resumes, keeping local variables alive between calls. That makes it perfect for running calculations where you feed values in over time and want an updated result each step, without a class.

WHERE IT IS USED

Streaming averages/counters power live dashboards and sensor-data smoothing.

HINT Keep `total` and `count` across iterations; `yield` the average inside a loop over the inputs.

6. Applied Mini-Problems

Small slices of real systems — ledgers, session logs, billing, sweep-lines, and parsers — that combine everything from Book 1 and this book.

Problem 6.1

HARD

Given a list of transaction dicts with 'type' ('credit'/'debit') and 'amount', compute the final balance starting from 0, and raise a custom `OverdraftError` if any debit would take the balance negative.

CONCEPT: COMBINING OOP EXCEPTIONS WITH ITERATION

This ties together Book 1's exception chapter with real iteration. A custom exception class documents the specific failure. Processing records in a loop while validating each against a running total is the shape of nearly every transaction-processing task.

WHERE IT IS USED

Ledger systems, wallets, and inventory counters all validate each operation against current state and reject invalid ones with meaningful errors.

HINT Define `class OverdraftError(Exception)`. Check before applying each debit.

Problem 6.2

HARD

You have a log of events as `(timestamp, user)` tuples. Find each user's **session count**, where a new session starts if more than 30 minutes passed since that user's previous event. Timestamps are integer minutes.

CONCEPT: STATEFUL GROUPING WITH A LAST-SEEN MAP

Track per-user state in a dictionary: the last timestamp seen and the running session count. For each event, compare the gap to the threshold to decide whether to start a new session. Sorting events first ensures chronological order.

WHERE IT IS USED

Exactly how analytics tools (web sessions, app opens) group raw events into meaningful visits.

HINT Sort by timestamp. Keep `last_seen[user]` and `sessions[user]` dicts.

Problem 6.3 — MEDIUM

Given a grid (list of lists) of 0s and 1s, count how many rows are **identical** to at least one other row.

CONCEPT: MAKING LISTS HASHABLE VIA TUPLES

Lists can't be dictionary keys because they're mutable, but **tuples can**. Converting each row to a tuple lets you count row frequencies in a dict. Rows appearing more than once are the duplicates.

WHERE IT IS USED

Duplicate-row detection is common in data cleaning and in spotting repeated patterns in matrices or images.

HINT Count tuple(row) frequencies, then sum the counts of rows that appear 2+ times.

Problem 6.4 — HARD

Parse a string of the form **"3a2b4c"** (count followed by character) and expand it into **"aaabbcccc"**. Assume counts can be multiple digits.

CONCEPT: PARSING MIXED DIGIT/LETTER STREAMS

When numbers and letters are interleaved, accumulate digit characters into a number string until you hit a non-digit, then act. This 'build up the number, then apply it' loop is the heart of simple parsers and tokenizers.

WHERE IT IS USED

Mini-parsers like this underlie template engines, simple markup expanders, and decoding compressed formats.

HINT Keep a `num` string; when you see a letter, repeat it `int(num)` times and reset.

Problem 6.5 — HARD

Given a list of words, return the **top k most frequent** words, ordered by frequency (descending) then alphabetically.

CONCEPT: COUNT THEN MULTI-KEY SORT, THEN SLICE

A three-step recipe: count with a dict, sort the items by a tuple key (frequency descending, word ascending), then take the first `k`. Combining a frequency counter with a multi-key sort solves a huge family of 'top-k' questions.

WHERE IT IS USED

'Top search terms', 'trending tags', and 'most common errors' dashboards are exactly this.

HINT `sorted(freq.items(), key=lambda kv:(-kv[1], kv[0]))[:k]`.

Problem 6.6

HARD

You are given start and end times of meetings as `(start, end)` pairs. Determine the **maximum number of meetings happening at the same time**.

CONCEPT: SWEEP LINE WITH +1/-1 EVENTS

Turn each meeting into two events: a `+1` at its start and a `-1` at its end. Sort all events by time (ends before starts at the same moment) and sweep through, tracking a running count. The peak of that count is the answer — no nested comparison of every pair needed.

WHERE IT IS USED

This 'sweep line' is how systems compute peak concurrent users, overlapping bookings, or maximum simultaneous calls.

HINT Build events `(time, +1/-1)`, sort, then track a running max.

Problem 6.7

MEDIUM

Given a dictionary of item → price and a shopping list (item → quantity), compute the total bill, applying a 10% discount to any single line whose subtotal exceeds 1000. Skip items not in the price list.

CONCEPT: DICTIONARY LOOKUPS WITH CONDITIONAL BUSINESS RULES

Real billing walks the order, looks each item up safely (skipping unknowns), computes a line subtotal, and applies conditional rules. Combining `.get()` guards with per-line conditionals mirrors how business logic is actually coded.

WHERE IT IS USED

This is a miniature checkout engine — the same shape powers carts, invoicing, and tax/discount rules.

HINT Loop the order; skip if item not in prices; apply the discount per line before adding.

SOLUTIONS

Answers, Code & Walkthroughs

> *Complete solutions for every problem, each with runnable code, its exact output, and a plain-English walkthrough of how and why it works. Solution numbers match problem numbers.*

S1. String Manipulation

Solution 1.1 MEDIUM

*Given a sentence, return it with the word order reversed but each word's letters intact. Example: "**code every day**" → "**day every code**".*

```
def reverse_words(sentence):
    words = sentence.split()      # ['code', 'every', 'day']
    return " ".join(words[::-1])  # 'day every code'

print(reverse_words("code every day"))
print(reverse_words("practice makes perfect"))
```

```
Output ▸ day every code
perfect makes practice
```

Walkthrough: `split()` breaks on whitespace into a word list. The slice `[::-1]` reverses that list (not the letters). `join` glues the words back with single spaces. Because we split first, extra spaces collapse automatically.

Solution 1.2 MEDIUM

*Check whether a string is a **pangram** — it contains every letter of the English alphabet at least once (case-insensitive).*

```
import string

def is_pangram(text):
    alphabet = set(string.ascii_lowercase)   # {'a'..'z'}
    letters = set(text.lower())
    return alphabet <= letters               # is alphabet a subset?

print(is_pangram("The quick brown fox jumps over a lazy dog"))
print(is_pangram("Hello world"))
```

```
Output ▸ True
False
```

Walkthrough: `string.ascii_lowercase` gives 'abcdefghijklmnopqrstuvwxyz'. We turn both it and the (lowercased) text into sets. `alphabet <= letters` asks whether every alphabet letter appears in the text — a clean subset check that beats looping 26 times.

Solution 1.3 HARD

*Compress a string using **run-length encoding**: replace runs of repeated characters with the character followed by its count. Example: "**aaabbc**" → "**a3b2c1**".*

```
def compress(s):
    if not s:
        return ""
    result = []
    prev = s[0]
    count = 1
    for ch in s[1:]:
        if ch == prev:
            count += 1
        else:
            result.append(prev + str(count))
            prev = ch
            count = 1
    result.append(prev + str(count))   # flush the last run
    return "".join(result)

print(compress("aaabbc"))
print(compress("abcd"))
```

```
Output ▸ a3b2c1
a1b1c1d1
```

Walkthrough: We seed `prev` with the first character and count 1. Each following character either extends the current run (increment count) or ends it (append `char+count`, reset). The crucial detail is the final `append` after the loop — the last run never hits an 'else', so it must be flushed manually.

Solution 1.4 MEDIUM

Given a string, find the character(s) with the highest frequency. If several tie, return them all in the order they first appear.

```
def most_frequent(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    top = max(freq.values())
    result = []
    for ch in s:                        # original order
        if freq[ch] == top and ch not in result:
            result.append(ch)
    return result

print(most_frequent("aabbbcc"))    # b appears most
print(most_frequent("xxyyz"))      # x and y tie
```

```
Output ▸ ['b']
['x', 'y']
```

Walkthrough: After counting, `max(freq.values())` is the winning count. We then walk the original string so ties come out in first-seen order, guarding with `ch not in result` so each character is added once. Iterating the string (not the dict) is what preserves order.

Solution 1.5 HARD

*Validate a string as a simple **identifier**: it must start with a letter or underscore, contain only letters, digits, or underscores, and not be a Python keyword. Return True/False.*

```
import keyword

def is_valid_identifier(name):
    if not name:
        return False
    first = name[0]
    if not (first.isalpha() or first == "_"):
        return False
    for ch in name[1:]:
        if not (ch.isalnum() or ch == "_"):
            return False
    if name in keyword.kwlist:          # reserved word?
        return False
    return True

for test in ["value1", "_temp", "2fast", "my-var", "class"]:
    print(test, "->", is_valid_identifier(test))
```

```
Output ▸ value1 -> True
_temp -> True
2fast -> False
my-var -> False
class -> False
```

Walkthrough: Each rule is its own guard clause returning `False` early: empty string, illegal first character, any illegal later character, or a reserved keyword. Only a name surviving all four checks returns `True`. Layered guards read top-to-bottom like the spec itself.

Solution 1.6 MEDIUM

*Given two strings, find the **longest common prefix** they share. Example: "**flower**", "**flight**" → "**fl**".*

```
def common_prefix(a, b):
    prefix = []
    for ca, cb in zip(a, b):
        if ca != cb:
            break
        prefix.append(ca)
    return "".join(prefix)

print(common_prefix("flower", "flight"))
print(common_prefix("dog", "cat"))
```

```
Output ▸ fl
```

Walkthrough: `zip` yields ('f','f'), ('l','l'), ('o','i')… We collect matching characters and stop at the first mismatch ('o' vs 'i'), giving 'fl'. For 'dog'/'cat' the very first pair differs, so the prefix is empty. `zip` also handles different lengths for free.

Solution 1.7 MEDIUM

*Determine whether one string is a **rotation** of another. Example: "**waterbottle**" is a rotation of "**erbottlewat**".*

```
def is_rotation(a, b):
    if len(a) != len(b):
        return False
    return b in (a + a)

print(is_rotation("waterbottle", "erbottlewat"))
print(is_rotation("hello", "lohel"))
print(is_rotation("abc", "acb"))
```

```
Output ▸ True
True
False
```

Walkthrough: Doubling 'waterbottle' gives 'waterbottlewaterbottle', which contains every possible rotation as a slice — including 'erbottlewat'. The length guard rejects mismatched sizes early. 'acb' isn't a rotation of 'abc', so the membership test fails.

Solution 1.8 MEDIUM

Given a string with words separated by spaces, return a dictionary mapping each word length to the list of words having that length.

```
def group_by_length(sentence):
    groups = {}
    for word in sentence.split():
        groups.setdefault(len(word), []).append(word)
    return groups

print(group_by_length("the cat sat on a mat today"))
```

```
Output ▸ {3: ['the', 'cat', 'sat', 'mat'], 2: ['on'], 1: ['a'], 5: ['today']}
```

Walkthrough: Each word is filed under its length. `setdefault` makes an empty list the first time a length appears, so 'the', 'cat', 'sat', 'mat' all collect under key 3. The computed key `len(word)` is what drives the grouping.

S2. Number Theory & Digit Math

Solution 2.1 MEDIUM

Given a positive integer, return the count of digits, the sum of digits, and the reversed number — without converting to a string.

```
def analyze(n):
    count = 0
    digit_sum = 0
    reversed_n = 0
    while n > 0:
        digit = n % 10          # last digit
        digit_sum += digit
        reversed_n = reversed_n * 10 + digit
        count += 1
        n = n // 10             # drop last digit
    return count, digit_sum, reversed_n

print(analyze(4825))
```

```
Output ▸ (4, 19, 5284)
```

Walkthrough: Each loop peels the last digit with `% 10`, adds it to the sum, and shifts it into `reversed_n` by multiplying the accumulator by 10 first. `// 10` then discards that digit. When `n` reaches 0 we have counted 4 digits, summed to 19, and rebuilt 5284.

Solution 2.2 MEDIUM

*Print the first **n** prime numbers (not primes up to n, but exactly n primes). Example for n=5: **2, 3, 5, 7, 11**.*

```
def is_prime(x):
    if x < 2:
        return False
    for i in range(2, int(x ** 0.5) + 1):
        if x % i == 0:
            return False
    return True

def first_n_primes(n):
    primes = []
    candidate = 2
    while len(primes) < n:
        if is_prime(candidate):
            primes.append(candidate)
        candidate += 1
    return primes

print(first_n_primes(5))
print(first_n_primes(8))
```

```
Output ▸ [2, 3, 5, 7, 11]
[2, 3, 5, 7, 11, 13, 17, 19]
```

Walkthrough: The `while` loop does not care about an upper limit — it simply keeps testing rising candidates until the list holds `n` primes. The square-root optimization inside `is_prime` keeps each test fast. Separating 'is it prime' from 'collect n of them' is the key design move.

Solution 2.3 HARD

*Convert an integer to its **binary representation** as a string, without using `bin()` or format specifiers.*

```
def to_binary(n):
    if n == 0:
        return "0"
    bits = []
    while n > 0:
        bits.append(str(n % 2))    # remainder is next bit
        n = n // 2
    return "".join(reversed(bits)) # digits came out backwards

print(to_binary(13))     # 1101
print(to_binary(0))      # 0
print(to_binary(255))    # 11111111
```

```
Output ▸ 1101
0
11111111
```

Walkthrough: Dividing by 2 and keeping remainders produces the binary digits from least to most significant, so they come out reversed. We collect them, then `reversed` + `join` orders them correctly. The `n == 0` guard matters because the loop never runs for zero.

Solution 2.4 MEDIUM

*A number is a **perfect number** if it equals the sum of its proper divisors (divisors excluding itself). Example: 6 = 1+2+3. Find all perfect numbers up to a limit.*

```
def divisor_sum(n):
    total = 0
    for i in range(1, n // 2 + 1):
        if n % i == 0:
            total += i
    return total

def perfect_numbers(limit):
    return [n for n in range(2, limit + 1) if divisor_sum(n) == n]

print(perfect_numbers(30))     # 6 and 28
print(divisor_sum(28))         # 1+2+4+7+14 = 28
```

```
Output ▸ [6, 28]
28
```

Walkthrough: `divisor_sum` adds every number up to `n//2` that divides `n` cleanly — no proper divisor can exceed half of n. The comprehension then keeps only numbers equal to their own divisor sum. For 28, 1+2+4+7+14 = 28, so it qualifies.

Solution 2.5 HARD

*Given an amount and a list of coin denominations, compute the **minimum number of coins** needed using a greedy approach (assume standard denominations like [1, 2, 5, 10, 20, 50]).*

```
def min_coins(amount, coins):
    coins = sorted(coins, reverse=True)   # largest first
    used = {}
    for coin in coins:
        if amount == 0:
            break
        count, amount = divmod(amount, coin)
        if count > 0:
            used[coin] = count
    return used, sum(used.values())

breakdown, total = min_coins(93, [1, 2, 5, 10, 20, 50])
print(breakdown)
print("Total coins:", total)
```

```
Output ▸ {50: 1, 20: 2, 2: 1, 1: 1}
Total coins: 5
```

Walkthrough: Sorting descending lets us grab the biggest coin first. `divmod` tells us how many of that coin fit (goes into the tally) and updates the leftover amount in a single call. Repeating down the list, 93 becomes 50 + 20 + 20 + 2 + 1 = five coins. Greedy works here because the denominations are 'canonical'.

Solution 2.6 MEDIUM

*Given two integers, compute their **LCM** (least common multiple) using their GCD.*

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)

print(lcm(4, 6))     # 12
print(lcm(21, 6))    # 42
```

```
Output ▸ 12
42
```

Walkthrough: Euclid's loop reduces (a,b) to (b, a%b) until b is 0, leaving the GCD. Dividing the product `a*b` by that GCD yields the LCM. For 4 and 6, gcd is 2, so lcm is 24//2 = 12. Deriving LCM from GCD avoids listing multiples.

Solution 2.7 MEDIUM

*Count how many integers from 1 to n are **divisible by 3 or 5**, and return their sum.*

```
def multiples_3_or_5(n):
    matches = [x for x in range(1, n + 1) if x % 3 == 0 or x % 5 == 0]
    return len(matches), sum(matches)

count, total = multiples_3_or_5(20)
print("Count:", count, "Sum:", total)
```

```
Output ▸ Count: 9 Sum: 98
```

Walkthrough: The comprehension keeps numbers divisible by 3 or 5: 3,5,6,9,10,12,15,18,20 — nine of them, summing to 98. Building the list once lets us report both the count and the sum without recomputing. The `or` captures either divisibility rule.

Solution 2.8 HARD

*Given a number, return its **Roman numeral** representation (valid up to 3999).*

```
def to_roman(num):
    values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]
    symbols = ["M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"]
    result = []
    for value, symbol in zip(values, symbols):
        while num >= value:
            result.append(symbol)
            num -= value
    return "".join(result)

print(to_roman(2024))    # MMXXIV
print(to_roman(49))      # XLIX
```

```
Output ▸ MMXXIV
XLIX
```

Walkthrough: We march through value/symbol pairs largest-first. For 2024: two 1000s (MM), then 10+10 (XX), then 4 as IV — MMXXIV. Including subtractive pairs like XL (40) and IX (9) in the tables means the same simple greedy loop handles the tricky cases without special-casing.

S3. Lists & Dictionaries

Solution 3.1 HARD

*Given a list of numbers, return a new list where each element is the product of all **other** elements (do not use division). Example: **[1,2,3,4]** → **[24,12,8,6]**.*

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):              # products to the left
        result[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):  # products to the right
        result[i] *= suffix
        suffix *= nums[i]
    return result

print(product_except_self([1, 2, 3, 4]))
print(product_except_self([2, 3, 4, 5]))
```

```
Output ▸ [24, 12, 8, 6]
[60, 40, 30, 24]
```

Walkthrough: After the first loop, `result[i]` holds the product of all elements before `i`. The second loop walks backward multiplying in the product of all elements after `i`. Each position ends up as left-product times right-product — every element except itself — in two linear passes, no division.

Solution 3.2 MEDIUM

Merge two dictionaries of category → list-of-items so shared categories have their lists combined (no duplicate items, order preserved).

```
def merge_grouped(d1, d2):
    result = {k: list(v) for k, v in d1.items()}   # deep-ish copy
    for key, items in d2.items():
        bucket = result.setdefault(key, [])
        for item in items:
            if item not in bucket:
                bucket.append(item)
    return result

a = {"fruit": ["apple", "pear"], "veg": ["carrot"]}
b = {"fruit": ["pear", "mango"], "grain": ["rice"]}
print(merge_grouped(a, b))
```

```
Output ▸ {'fruit': ['apple', 'pear', 'mango'], 'veg': ['carrot'], 'grain': ['rice']}
```

Walkthrough: We start with copies of the first dict's lists so we never mutate the input. For each category in the second dict, `setdefault` hands us the existing list or a fresh one. The inner check appends only unseen items, so 'pear' isn't duplicated while 'mango' and the new 'grain' category are added.

Solution 3.3 MEDIUM

Given a list of (name, score) tuples, return the names sorted by score descending, and for equal scores, alphabetically by name.

```
def rank(players):
    ordered = sorted(players, key=lambda p: (-p[1], p[0]))
    return [name for name, score in ordered]

data = [("Ravi", 90), ("Asha", 90), ("Meena", 85), ("Karan", 90)]
print(rank(data))
```

```
Output ▸ ['Asha', 'Karan', 'Ravi', 'Meena']
```

Walkthrough: The key `(-score, name)` means: sort by negative score so higher scores come first, and break ties by name in normal ascending order. The three 90s (Asha, Karan, Ravi) sort alphabetically among themselves, then Meena's 85 follows. Tuple keys handle 'sort by A then B' with no extra code.

Solution 3.4 MEDIUM

*Rotate a list to the right by **k** positions. Example: **[1,2,3,4,5]** rotated by 2 → **[4,5,1,2,3]**.*

```
def rotate_right(lst, k):
    if not lst:
        return lst
    k = k % len(lst)            # handle k larger than list
    return lst[-k:] + lst[:-k]

print(rotate_right([1, 2, 3, 4, 5], 2))
print(rotate_right([1, 2, 3, 4, 5], 7))   # 7 % 5 == 2
```

```
Output ▸ [4, 5, 1, 2, 3]
[4, 5, 1, 2, 3]
```

Walkthrough: `lst[-k:]` grabs the last k items and `lst[:-k]` grabs the rest; concatenating puts the tail in front. The modulo step means a rotation of 7 on a 5-item list behaves like a rotation of 2, avoiding out-of-range slicing.

Solution 3.5 MEDIUM

*Given a list of integers and a target sum, return the first **pair** of values that add up to the target. Return None if no pair exists.*

```
def find_pair(nums, target):
    seen = set()
    for num in nums:
        complement = target - num
        if complement in seen:
            return (complement, num)
        seen.add(num)
    return None

print(find_pair([4, 2, 7, 1, 9], 11))   # 4 + 7
print(find_pair([1, 2, 3], 100))
```

```
Output ▸ (4, 7)
None
```

Walkthrough: As we scan, `seen` holds every earlier value. For 7 the complement is 4, which is already in `seen`, so we return (4, 7) immediately. This single-pass set lookup replaces a nested double loop, and membership tests on a set are effectively instant.

Solution 3.6 MEDIUM

*Given a list, return a dictionary mapping each element to a list of the **indexes** where it appears. Example: **[a,b,a,c,b] → {a: [0,2], b:[1,4], c:[3]}**.*

```
def index_map(lst):
    result = {}
    for i, value in enumerate(lst):
        result.setdefault(value, []).append(i)
    return result

print(index_map(["a", "b", "a", "c", "b"]))
```

```
Output ▸ {'a': [0, 2], 'b': [1, 4], 'c': [3]}
```

Walkthrough: `enumerate` hands us (0,'a'), (1,'b'), (2,'a')… `setdefault` ensures each value starts with an empty list, then we append the current index. 'a' collects indexes 0 and 2, 'b' collects 1 and 4. The result is a map from value to all its positions.

Solution 3.7 HARD

*Given two lists that may contain duplicates, return their **intersection** preserving how many times each element appears in both. Example: **[1,2,2,3]** and **[2,2,4]** → **[2,2]**.*

```
def multiset_intersection(a, b):
    def counts(lst):
        c = {}
        for x in lst:
            c[x] = c.get(x, 0) + 1
        return c
    ca, cb = counts(a), counts(b)
    result = []
    for key in ca:
        if key in cb:
            result.extend([key] * min(ca[key], cb[key]))
    return result

print(multiset_intersection([1, 2, 2, 3], [2, 2, 4]))
print(multiset_intersection([1, 1, 1], [1, 1]))
```

```
Output ▸ [2, 2]
[1, 1]
```

Walkthrough: Counting gives {1:1,2:2,3:1} and {2:2,4:1}. Only key 2 is shared; `min(2,2)` is 2, so we emit 2 twice. For the second case, 1 appears three times in one list and twice in the other, so `min(3,2)` yields two 1s. Taking the minimum count is the whole trick.

S4. Matrices & Grids

Solution 4.1 MEDIUM

*Given a 2D grid (list of lists), return its **transpose** (rows become columns) without using external libraries.*

```
def transpose(grid):
    return [list(col) for col in zip(*grid)]

matrix = [[1, 2, 3],
          [4, 5, 6]]
for row in transpose(matrix):
    print(row)
```

```
Output ▸ [1, 4]
[2, 5]
[3, 6]
```

Walkthrough: `zip(*matrix)` is `zip([1,2,3], [4,5,6])`, which yields (1,4), (2,5), (3,6) — the columns. Wrapping each in `list` turns the tuples back into rows. A 2x3 grid becomes 3x2, and the splat operator does the heavy lifting.

Solution 4.2 MEDIUM

*Given a square matrix, compute the sum of both **diagonals** (count the center once if the size is odd).*

```
def diagonal_sum(m):
    n = len(m)
    total = 0
    for i in range(n):
        total += m[i][i]            # main diagonal
        total += m[i][n - 1 - i]    # anti-diagonal
    if n % 2 == 1:                  # center counted twice
        total -= m[n // 2][n // 2]
    return total

matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]
print(diagonal_sum(matrix))
```

```
Output ▸ 25
```

Walkthrough: Main diagonal 1+5+9 and anti-diagonal 3+5+7 overlap at the center 5. Adding both gives 30, then subtracting the double-counted center (5) yields 25. The index formulas `m[i][i]` and `m[i][n-1-i]` pick the two diagonals directly.

Solution 4.3 MEDIUM

*Given a grid of 0s and 1s, find the row with the **most** 1s and return its index (first such row on ties).*

```
def row_most_ones(grid):
    best_count = -1
    best_index = -1
    for i, row in enumerate(grid):
        ones = sum(row)
        if ones > best_count:      # strict > keeps first on ties
            best_count = ones
            best_index = i
    return best_index

grid = [[0, 1, 1],
        [1, 1, 1],
        [0, 0, 1]]
print(row_most_ones(grid))
```

```
Output ▸ 1
```

Walkthrough: `sum(row)` gives 2, 3, 1 for the three rows. We keep the running best, updating only when a row strictly beats it, so row 1 (three 1s) wins. Using strict `>` means an equal later row wouldn't displace an earlier one — that's how we return the first on ties.

Solution 4.4 HARD

*Rotate a square matrix 90° **clockwise** in place-style (return a new grid). Example first row becomes the last column.*

```
def rotate_90(m):
    transposed = [list(row) for row in zip(*m)]
    return [row[::-1] for row in transposed]

matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]
for row in rotate_90(matrix):
    print(row)
```

```
Output ▸ [7, 4, 1]
[8, 5, 2]
[9, 6, 3]
```

Walkthrough: Transposing turns rows into columns: row 0 becomes [1,4,7]. Reversing each transposed row then places the bottom-left at the top-right, completing the clockwise turn — [1,4,7] becomes [7,4,1]. Two familiar steps compose into the rotation.

Solution 4.5 HARD

*Given a grid, check whether it is a **magic square**: all rows, all columns, and both diagonals sum to the same value.*

```
def is_magic(m):
    n = len(m)
    sums = []
    sums.extend(sum(row) for row in m)          # rows
    sums.extend(sum(col) for col in zip(*m))    # columns
    sums.append(sum(m[i][i] for i in range(n))) # main diag
    sums.append(sum(m[i][n-1-i] for i in range(n)))  # anti diag
    return len(set(sums)) == 1

square = [[2, 7, 6],
          [9, 5, 1],
          [4, 3, 8]]
print(is_magic(square))
print(is_magic([[1, 2], [3, 4]]))
```

```
Output ▸ True
False
```

Walkthrough: Every row, column, and diagonal of the first grid sums to 15, so the set of sums is {15} — size 1, magic. `zip(*m)` gives the columns cleanly. The second grid produces several different sums, so the set has more than one element and the check fails.

S5. Functions, Recursion & Beyond

Solution 5.1 HARD

*Write a function that flattens an arbitrarily nested list of integers into a single flat list, to **any** depth. Example: **[1,[2,[3,[4]]],5]** → **[1,2,3,4,5]**.*

```
def flatten(nested):
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.extend(flatten(item))   # recurse into sub-list
        else:
            flat.append(item)
    return flat

print(flatten([1, [2, [3, [4]]], 5]))
print(flatten([[1, 2], [3, [4, [5, 6]]]]))
```

```
Output ▸ [1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 6]
```

Walkthrough: For each item we ask: is it a list? If yes, we call `flatten` on it and `extend` our result with whatever comes back (already flat). If no, we append the value. Because the function calls itself on sub-lists, it unwinds nesting of any depth without knowing the depth in advance.

Solution 5.2 HARD

*Implement a simple **memoized** Fibonacci using a dictionary cache so repeated calls are fast. Show that fib(50) returns instantly.*

```
def fib(n, cache={}):
    if n <= 1:
        return n
    if n in cache:              # already computed?
        return cache[n]
    cache[n] = fib(n - 1, cache) + fib(n - 2, cache)
    return cache[n]

print(fib(10))
print(fib(50))
```

```
Output ▸ 55
12586269025
```

Walkthrough: Before doing any work for `n`, we check the cache. If it's there, we return instantly; otherwise we compute it once, store it, and return. Because each Fibonacci value is calculated a single time, `fib(50)` finishes immediately instead of making billions of calls.

Solution 5.3 HARD

Write a function `compose(*funcs)` that returns a new function applying the given functions right-to-left. Example: `compose(double, increment)(5)` computes double(increment(5)) = 12.

```
def compose(*funcs):
    def composed(x):
        for f in reversed(funcs):     # right-to-left
            x = f(x)
        return x
    return composed

double = lambda n: n * 2
increment = lambda n: n + 1

pipeline = compose(double, increment)
print(pipeline(5))                    # double(increment(5)) = 12
print(compose(increment, double)(5))  # increment(double(5)) = 11
```

```
Output ▸ 12
11
```

Walkthrough: `compose` returns `composed`, which applies the functions from right to left. For `compose(double, increment)(5)`: increment runs first (5→6), then double (6→12). Swapping the order changes the result, showing the pipeline direction. Returning a function from a function is the core idea.

Solution 5.4 MEDIUM

Build a function that takes a variable number of keyword arguments and returns them formatted as a query string. Example: `query(page=2, sort="name")` → ***"page=2&sort=name"***.

```
def query(**kwargs):
    parts = [f"{key}={value}" for key, value in kwargs.items()]
    return "&".join(parts)

print(query(page=2, sort="name"))
print(query(q="python", limit=10, page=1))
```

```
Output ▸ page=2&sort=name
q=python&limit=10&page=1
```

Walkthrough: `**kwargs` captures every named argument as a dict entry. The comprehension formats each pair as `key=value`, and `'&'.join` stitches them with ampersands. Because dicts preserve insertion order in modern Python, the parameters appear in the order given.

Solution 5.5 HARD

*Write a **decorator** that times how long a function takes and prints it, then returns the function's result unchanged.*

```
import time

def timed(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} ran (result below)")
        return result
    return wrapper

@timed
def add(a, b):
    return a + b

print(add(3, 4))
```

```
Output ▸ add ran (result below)
7
```

Walkthrough: `@timed` replaces `add` with `wrapper`. When called, the wrapper records a start time, runs the real `add`, measures elapsed time, prints a note, and returns the untouched result (7). `*args, **kwargs` let the wrapper work for any function signature.

Solution 5.6 HARD

*Implement a function that returns a **running average** generator: each time you send it a number, it yields the average of all numbers seen so far.*

```
def running_average(numbers):
    total = 0
    count = 0
    for n in numbers:
        total += n
        count += 1
        yield total / count      # average so far

for avg in running_average([10, 20, 30, 40]):
    print(round(avg, 2))
```

```
Output ▸ 10.0
15.0
20.0
25.0
```

Walkthrough: Each iteration adds the new number to `total`, bumps `count`, and `yield`s the current average — then pauses until the next value. After 10 the average is 10; after 10,20 it's 15; and so on. The generator remembers `total` and `count` between yields, so no external state is needed.

S6. Applied Mini-Problems

Solution 6.1 HARD

Given a list of transaction dicts with 'type' ('credit'/'debit') and 'amount', compute the final balance starting from 0, and raise a custom `OverdraftError` if any debit would take the balance negative.

```
class OverdraftError(Exception):
    pass

def process(transactions):
    balance = 0
    for t in transactions:
        if t["type"] == "credit":
            balance += t["amount"]
        elif t["type"] == "debit":
            if t["amount"] > balance:
                raise OverdraftError(
                    f"Debit {t['amount']} exceeds balance {balance}")
            balance -= t["amount"]
    return balance

txns = [{"type": "credit", "amount": 500},
        {"type": "debit", "amount": 200},
        {"type": "debit", "amount": 400}]
try:
    print(process(txns))
except OverdraftError as e:
    print("Blocked:", e)
```

```
Output ▸ Blocked: Debit 400 exceeds balance 300
```

Walkthrough: We walk the transactions updating `balance`. Credits add; debits first check whether they'd overdraw, and if so raise `OverdraftError` with a clear message. The 400 debit fails against a balance of 300, so the `except` block reports exactly why — validation plus custom exceptions working together.

Solution 6.2 HARD

*You have a log of events as `(timestamp, user)` tuples. Find each user's **session count**, where a new session starts if more than 30 minutes passed since that user's previous event. Timestamps are integer minutes.*

```
def count_sessions(events, gap=30):
    events = sorted(events)          # chronological
    last_seen = {}
    sessions = {}
    for time, user in events:
        if user not in last_seen or time - last_seen[user] > gap:
            sessions[user] = sessions.get(user, 0) + 1
        last_seen[user] = time
    return sessions

log = [(0, "a"), (10, "a"), (50, "a"), (5, "b"), (100, "b")]
print(count_sessions(log))
```

```
Output ▸ {'a': 2, 'b': 2}
```

Walkthrough: After sorting, we process events in time order. A user gets a new session when we've never seen them or the gap since their last event exceeds 30 minutes. User 'a' has events at 0,10 (one session) then 50 (>30 gap, second session). 'b' jumps 5→100, also two sessions. Two small dicts hold all the state we need.

Solution 6.3 MEDIUM

*Given a grid (list of lists) of 0s and 1s, count how many rows are **identical** to at least one other row.*

```
def duplicate_rows(grid):
    counts = {}
    for row in grid:
        key = tuple(row)            # tuples are hashable
        counts[key] = counts.get(key, 0) + 1
    total = 0
    for count in counts.values():
        if count > 1:
            total += count          # all copies count
    return total

grid = [[1, 0, 1],
        [0, 0, 0],
        [1, 0, 1],
        [1, 1, 1],
        [0, 0, 0]]
print(duplicate_rows(grid))
```

```
Output ▸ 4
```

Walkthrough: Each row becomes a tuple so it can be a dict key. After counting, `[1,0,1]` appears twice and `[0,0,0]` appears twice, so 2 + 2 = 4 rows are duplicated. The single unique row `[1,1,1]` is excluded. Converting to tuples is the move that makes rows countable.

Solution 6.4 HARD

*Parse a string of the form **"3a2b4c"** (count followed by character) and expand it into **"aaabbcccc"**. Assume counts can be multiple digits.*

```
def expand(s):
    result = []
    num = ""
    for ch in s:
        if ch.isdigit():
            num += ch               # accumulate multi-digit count
        else:
            result.append(ch * int(num))
            num = ""                # reset for next group
    return "".join(result)

print(expand("3a2b4c"))
print(expand("10x2y"))
```

```
Output ▸ aaabbcccc
xxxxxxxxxxyy
```

Walkthrough: We collect digit characters into `num` so multi-digit counts like '10' work. On reaching a letter, we repeat it `int(num)` times, add it to the result, and reset `num` for the next count. This handles '10x' correctly by treating '1' and '0' as one number, not two.

Solution 6.5 HARD

*Given a list of words, return the **top k most frequent** words, ordered by frequency (descending) then alphabetically.*

```
def top_k_words(words, k):
    freq = {}
    for w in words:
        freq[w] = freq.get(w, 0) + 1
    ordered = sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))
    return [word for word, _ in ordered[:k]]

text = "apple banana apple cherry banana apple cherry".split()
print(top_k_words(text, 2))
```

```
Output ▸ ['apple', 'banana']
```

Walkthrough: Counts are apple:3, banana:2, cherry:2. Sorting by `(-count, word)` puts apple first, then breaks the banana/cherry tie alphabetically — 'banana' comes before 'cherry'. Slicing `[:2]` takes the top two. The negative count gives descending frequency while the word stays ascending, which is exactly the tie-break rule the problem asks for.

Solution 6.6 HARD

*You are given start and end times of meetings as `(start, end)` pairs. Determine the **maximum number of meetings happening at the same time**.*

```python
def max_overlap(meetings):
    events = []
    for start, end in meetings:
        events.append((start, 1))   # meeting begins
        events.append((end, -1))    # meeting ends
    events.sort(key=lambda e: (e[0], e[1]))  # -1 before +1 on ties
    current = 0
    peak = 0
    for _time, delta in events:
        current += delta
        peak = max(peak, current)
    return peak

meetings = [(1, 4), (2, 5), (7, 9), (3, 6)]
print(max_overlap(meetings))
```

```
Output ▸ 3
```

Walkthrough: Each meeting contributes a +1 start and -1 end event. Sorting with ends before starts at equal times prevents overcounting. Sweeping left to right, the count climbs as meetings 1,4 / 2,5 / 3,6 overlap around time 3, peaking at 3 simultaneous meetings before they end.

Solution 6.7 MEDIUM

Given a dictionary of item → price and a shopping list (item → quantity), compute the total bill, applying a 10% discount to any single line whose subtotal exceeds 1000. Skip items not in the price list.

```
def total_bill(prices, order):
    total = 0
    for item, qty in order.items():
        if item not in prices:
            continue                        # unknown item
        subtotal = prices[item] * qty
        if subtotal > 1000:
            subtotal *= 0.9                 # 10% off big lines
        total += subtotal
    return round(total, 2)

prices = {"pen": 20, "laptop": 600, "book": 150}
order = {"pen": 3, "laptop": 2, "book": 4, "eraser": 5}
print(total_bill(prices, order))
```

```
Output ▸ 1740.0
```

Walkthrough: Pens: 20×3 = 60 (under 1000, no discount). Laptops: 600×2 = 1200, which exceeds 1000, so 10% off → 1080. Books: 150×4 = 600 (no discount). 'eraser' isn't in `prices`, so `continue` skips it entirely. The total is 60 + 1080 + 600 = 1740.0. Rounding guards against float noise, and the per-line conditional is the business rule in action.

APPENDIX

Pattern Playbook

P. Reusable Patterns You Just Practiced

Every problem in this book was an instance of a small number of reusable patterns. Recognize the pattern, and the problem solves itself. Keep this page handy.

Pattern	When to reach for it	Core move
Frequency counter	Counting occurrences, finding modes/duplicates	`d[k] = d.get(k,0)+1`
Seen-set / complement	Find pairs, detect repeats in one pass	`if target-x in seen`
Prefix / suffix passes	"Everything except me" aggregates	Two directional sweeps
Digit math	Reverse, sum digits, base convert	`n%10` and `n//10`
Run / group tracking	Compress, sessionize, parse streams	Remember `prev` + count
Multi-key sort	Rank by A then B	`key=lambda x:(-x.a, x.b)`
Recursion for depth	Nested/unknown-depth structures	Base case + self-call
Memoization	Expensive repeated computation	Cache dict, check first
Tuple-as-key	Make lists/rows hashable & countable	`tuple(row)`
Layered validation	Rule-by-rule input checking	Guard clauses, early return

C. Final Coding-Round Checklist

1. **Identify the pattern first.** Ask "have I seen this shape before?" before writing code — the playbook above covers most rounds.
2. **State your approach out loud** before typing. Interviewers score reasoning.
3. **Handle the empty / single / duplicate cases.** They are where most solutions break.
4. **Prefer a clear pass over a clever nested loop.** A seen-set or a dict usually removes the inner loop.
5. **Name things well** — `seen`, `freq`, `prefix` — so your logic reads itself.
6. **Trace one example by hand** before saying "done".
7. **Wrap risky parsing in try/except** when input is untrusted.

M. Common Mistakes to Avoid

These are the errors that quietly cost points in timed rounds. Recognizing them is half the battle.

Mistake	Why it bites	Fix
Forgetting `input()` returns a string	Math on text raises `TypeError`	`int(input())`
Mutating a list while looping it	Skips elements or errors	Loop a copy or build a new list
Using a list as a dict key	Lists are unhashable	Convert with `tuple(row)`
Off-by-one in `range`	`stop` is excluded	Use `range(1, n+1)` for 1..n
Comparing floats with `==`	Rounding noise (0.1+0.2)	Round, or compare a small tolerance
Forgetting to flush the last run	Loops that act "on change" miss the final group	Append the final group after the loop
Bare `except:`	Hides real bugs	Catch the specific exception
Default mutable argument surprises	Shared state across calls	Understand it (as memo caches use) or use None

I. Pythonic Idioms Worth Memorizing

Short, expressive snippets that make solutions cleaner and faster to write under time pressure.

```python
# Swap without a temp
a, b = b, a

# Reverse a sequence
s[::-1]

# Conditional expression
label = "even" if n % 2 == 0 else "odd"

# Build a dict from two lists
dict(zip(keys, values))

# Count items in one line
from collections import Counter        # or use get(k,0)+1
Counter(items)

# Transpose a grid
list(zip(*grid))

# Flatten one level
[x for row in grid for x in row]

# Multi-key sort (A desc, B asc)
sorted(data, key=lambda x: (-x[0], x[1]))

# Membership over range with a condition
[x for x in range(n) if x % 3 == 0]

# Safe dict access with default
value = d.get(key, fallback)

# Group into buckets
d.setdefault(key, []).append(item)

# Unpack with a "rest"
first, *middle, last = my_list
```

Note: `collections.Counter` is standard-library, not a DSA topic — it is just a convenience dict for counting. The manual `get(k, 0) + 1` pattern works identically if you prefer to show the logic.

KEEP GOING

You now have two books of patterns and over 75 solved problems between them. Re-solve the ones that felt hard until the approach comes instantly — that automatic recall is exactly what a timed round rewards. Match the shape, apply the pattern, trace one example. You are ready.